

Cubes, Rollups, and Materialized Views and Tables

Objectives

After watching this video, you will be able to:

- Relate what a data cube is in terms of star schema
- Discuss the terms slice, dice, drill up or down, roll up, and pivot in terms of data cubes
- Describe what a materialized view is
- Recall two use cases for materialized views

Sure! Let's talk about **data cubes** in simple terms.

A data cube is like a multi-dimensional box that helps us organize and analyze data. Imagine you have a box of different colored balls, where each color represents a different product, and the balls are arranged by the year they were sold and the state they were sold in. Each ball's position in the box tells you how many of that product were sold in that year and state. This way, you can easily see and compare sales data across different dimensions, like products, years, and locations.

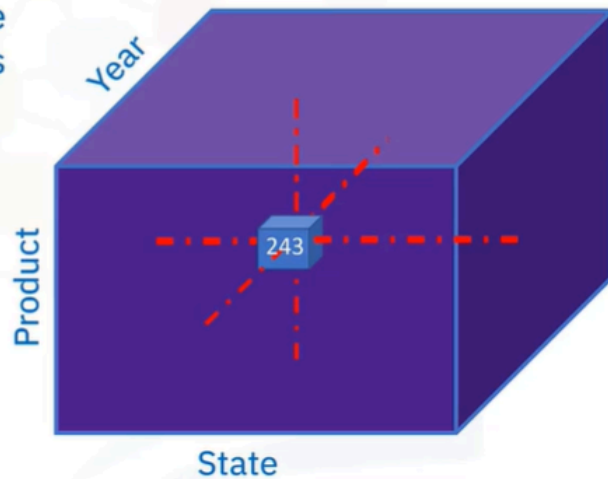
Now, think of operations you can perform on this box of balls. You can "slice" the box to look at just one year, like only 2018, or "dice" it to focus on specific products, like just gloves and t-shirts. You can also "drill down" to see more details about a specific product category, or "roll up" to get a summary of total sales. This makes it easier to understand and analyze the data from different perspectives!

What is a data cube?

- Example: Sales OLAP cube
- Coordinates = dimensions
- Cells = facts

Cube operations:

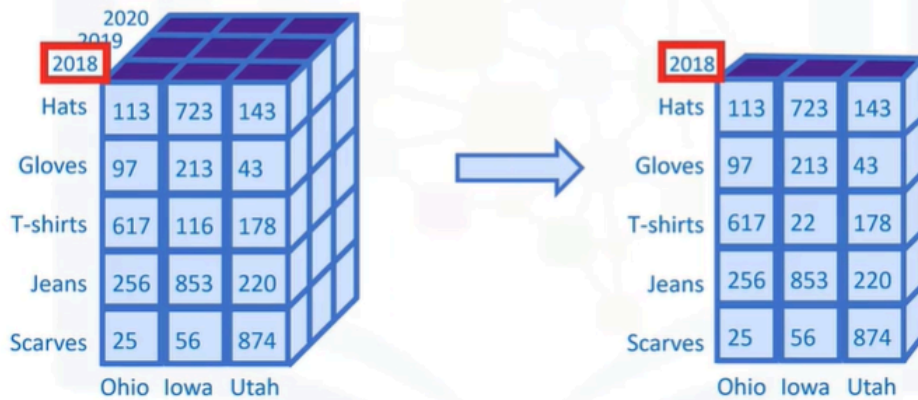
- Slicing
- Dicing
- Drilling up and down
- Pivoting
- Rolling up



- Let's use an example to illustrate the concept of a data cube. Here is a cube generated from an imaginary star schema for a Sales OLAP (online analytical processing system). The coordinates of the cube are defined by a set of dimensions, which are selected from the star schema. In this illustration, we are only showing three dimensions, but data cubes can have many dimensions. We have the Product categories corresponding to the items sold, the State or Province the items were sold from, and the Year these products were sold in.
- The cells of the cube are defined by a fact of interest from the schema, which could be something like "total sales in thousands of dollars." Here the "243" indicates "243 thousand dollars" for some given Product, State, and Year combination. There are many operations you can perform on data cubes, such as slicing, dicing, drilling up and down, pivoting, and rolling up.

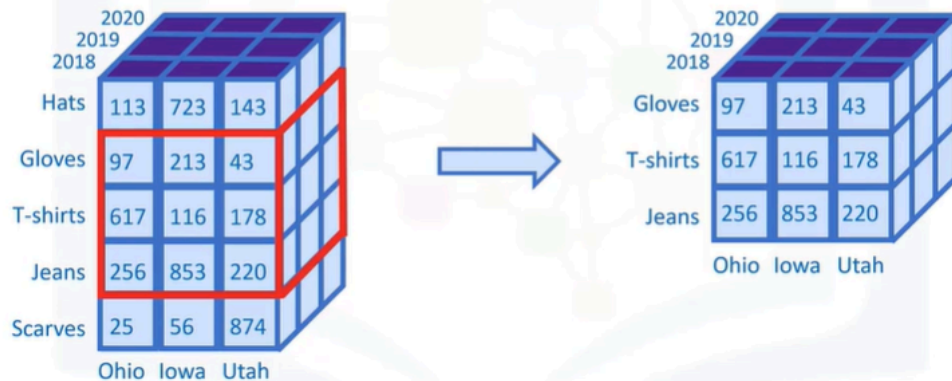
Slicing data cubes

Slicing reduces cube dimension by 1:



Dicing data cubes

Dicing shrinks a dimension:

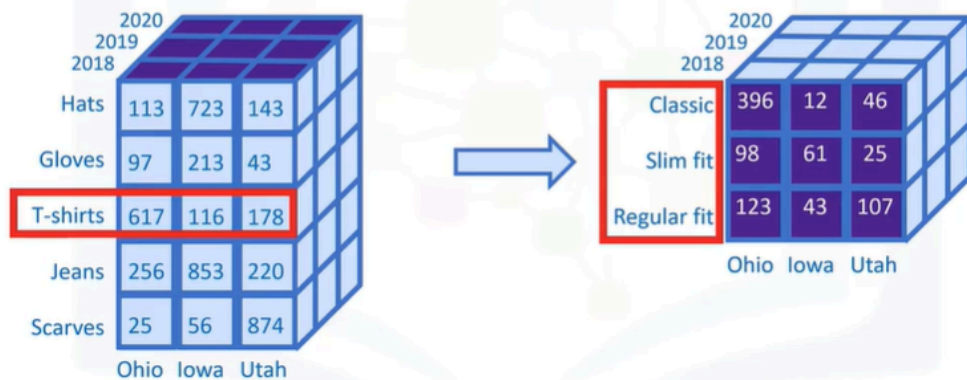


- Let's go over some examples of these operations, starting with slicing. Slicing a data cube involves selecting a single member from a dimension, which yields a data cube that has one dimension less than the original. For example, you can slice this sales cube by selecting only the year 2018 from the year dimension, allowing you to analyze sales totals for all sales states and all products for the year 2018.

- Similarly, dicing a cube involves selecting a subset of values from a dimension, effectively shrinking it. For example, you can dice this sales cube by selecting only "Gloves", "T-shirts", and "Jeans" from the Product-Type dimension, allowing you to restrict your view to just those product types.

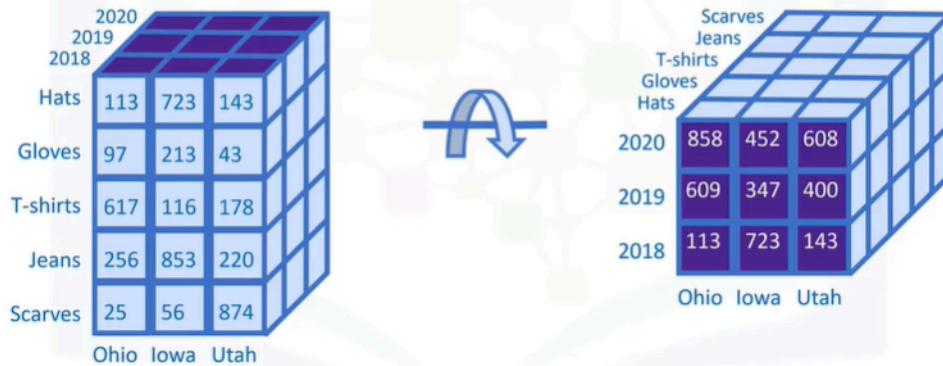
Drilling up or down in data cubes

Drilling into subcategories within a dimension:



- In snowflake schema, you will find hierarchies, or subcategories within some of your dimensions that you can drill into. Thus, for example, you can "drill down" into a particular member of the "Product category" dimension, such as "T-shirts," resulting in this view, which may include more specific "product groups" such as "Classic," "Slim fit," and "Regular fit." Drilling up is just the reverse process, which would take you back to the original data cube.

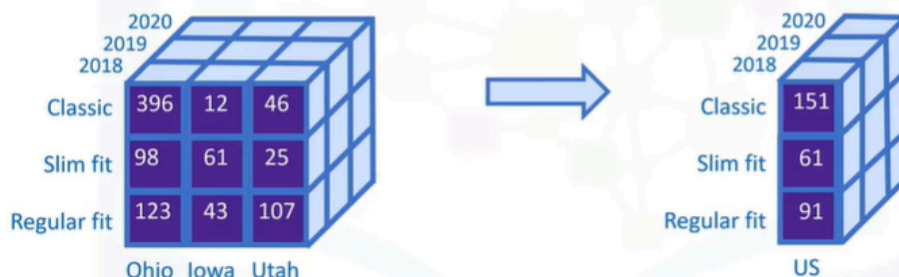
Pivoting data cubes



- Pivoting data cubes is straightforward. It involves a rotation of the data cube. In this case, the year and product dimensions have been interchanged, while the State dimension has been fixed "as is." Pivoting doesn't change its information content; it just changes the point of view you may choose to analyze it from.

Rolling up in data cubes

- Roll up = summarize a dimension
- Aggregate using COUNT, MIN, MAX, SUM, AVERAGE



- Rolling up means summarizing along a dimension. You can roll up a dimension by applying aggregations, such as COUNT, MIN, MAX, SUM, and AVERAGE. For example, you could calculate the average selling price of Classic, Slim fit, and Regular fit T-shirts by summing horizontally over the three US states and dividing by three

Materialized views

- A “snapshot” containing results of a query
 - Used to replicate data in a staging database, or
 - Precompute expensive queries for a data warehouse
 - Automatically keep query results synced to database
 - Safely work without affecting source database
-
- A “materialized view” is essentially a local, read-only copy, or snapshot, of the results of a query. They can be used to replicate data, for example to be used in a staging database as part of an ETL process, or to precompute and cache expensive queries, such as joins or aggregations, for use in data analytics environments. Materialized views also have options for automatically refreshing the data, thus keeping your query up-to-date. Because materialized views can be queried, you can safely work with them without worrying about affecting the source database.

Materialized views

Can be set up for different refresh options:

- Never - populated on creation only
- Upon request - manually or scheduled
- Immediately - automatically, after every statement

- Materialized Views can be set up to have different refresh options, such as:
Never: they are only populated when created, which is useful if the data seldom changes. Upon request: manually refresh, for example, after changes to the data have been made, or scheduled refresh, for example, after daily data loads. Immediately: automatically refresh after every statement.

Materialized view in PostgreSQL

Create a materialized view in PostgreSQL:

```
CREATE MATERIALIZED VIEW MY_MAT_VIEW  
[ WITH (storage_parameter [= value] [, ... ]) ]  
[ TABLESPACE tablespace_name ]  
AS SELECT * FROM <table_name>;  
  
REFRESH MATERIALIZED VIEW MY_MAT_VIEW;
```

- Here is how you might create a materialized view in PostgreSQL to replicate a table. Start by creating a "materialized view" object called "My underscore Mat underscore View", Specify some parameters, Specify the source

tablespace, say "tablespace underscore name", and Select all rows and columns from "table underscore name." In PostgreSQL you can only refresh materialized views manually, using the "refresh material view" command.

Summary

In this video, you learned that:

- A data cube represents a schema's dimensions as coordinates plus a fact for its values
- Many operations can be applied to data cubes, such as drilling down into hierarchical dimensions, slicing, dicing, and rolling up
- Materialized views can be used to replicate data or to precompute expensive queries
- Some tools can automatically refresh your material views for you

what is the aim of cube

The aim of a **data cube** is to facilitate efficient data analysis and reporting by organizing data in a multi-dimensional format. Here are the key objectives:

- **Multi-Dimensional Analysis:** Data cubes allow users to analyze data across multiple dimensions (e.g., products, time, location) simultaneously, making it easier to identify trends and patterns.
- **Simplified Queries:** They enable complex queries to be executed more simply and quickly, as the data is pre-aggregated and structured.
- **Enhanced Data Visualization:** Data cubes support better visualization of data, helping users to see relationships and insights that might be missed in traditional two-dimensional tables.

- **Efficient Data Retrieval:** By summarizing data in a cube format, it reduces the time needed to retrieve and analyze large datasets.

Overall, data cubes help users make informed decisions based on comprehensive and easily accessible data insights.

What is a data cube in a star schema?

A **data cube in a star schema** is a way to organize and analyze data that is structured in a star-like format. Here's a breakdown of the concept:

- **Star Schema Structure:** In a star schema, there is a central **fact table** that contains quantitative data (like sales amounts) and is surrounded by **dimension tables** that provide context (like product details, time periods, and locations). The fact table is the "center" of the star, while the dimension tables are the "points" of the star.
- **Data Cube Representation:** A data cube represents this star schema by using the dimensions from the dimension tables as axes. For example, if you have dimensions for **Product**, **Time**, and **Location**, the data cube will have these dimensions as its coordinates.
- **Cells of the Cube:** Each cell in the data cube contains aggregated data (like total sales) for a specific combination of the dimensions. For instance, one cell might represent total sales for "T-shirts" sold in "California" during "2020."
- **Operations:** You can perform various operations on the data cube, such as slicing (looking at one specific dimension), dicing (focusing on a subset of data), rolling up (aggregating data), and drilling down (getting more detailed data).

In summary, a data cube in a star schema helps users analyze and visualize data efficiently by organizing it in a way that highlights relationships between different dimensions.

How can you apply data cubes in data analysis?

You can apply **data cubes in data analysis** in several effective ways:

1. **Slicing:**

- This involves selecting a single dimension to focus on. For example, if you have a data cube with dimensions for **Product**, **Time**, and **Location**, you can slice the cube to view sales data for just one year, like 2020. This helps in analyzing trends for that specific time period.

2. **Dicing:**

- Dicing allows you to create a smaller cube by selecting specific values from multiple dimensions. For instance, you could dice the cube to view only sales data for "T-shirts" and "Jeans" in "California" during "2020." This narrows down the data for more targeted analysis.

3. **Drilling Down:**

- This operation lets you explore more detailed data within a dimension. For example, if you start with total sales for a product category, you can drill down to see sales for individual products within that category, providing deeper insights.

4. **Rolling Up:**

- Rolling up summarizes data along a dimension. For example, you could roll up sales data from individual months to see total sales for each quarter. This helps in understanding overall performance over time.

5. **Pivoting:**

- Pivoting involves rotating the data cube to view it from different perspectives. For example, you can switch the axes to analyze sales by product instead of by time. This flexibility allows for various analytical viewpoints.

6. **Aggregating Data:**

- Data cubes allow for quick aggregation of data using functions like SUM, AVERAGE, or COUNT. This is useful for generating reports and dashboards

that summarize key metrics.

By using these operations, data cubes enable analysts to gain insights, identify trends, and make data-driven decisions efficiently.

What is a data cube in data analysis?

A **data cube in data analysis** is a multi-dimensional array of data that allows for efficient organization, retrieval, and analysis of large datasets. Here are the key features:

- **Multi-Dimensional Structure:** A data cube organizes data across multiple dimensions, such as time, location, and product categories. Each dimension represents a different aspect of the data, allowing for complex analysis.
- **Cells Containing Data:** Each cell in the cube holds aggregated data values, such as totals or averages, for a specific combination of dimension values. For example, a cell might represent total sales for a specific product in a particular region during a certain time period.
- **Facilitates Analysis:** Data cubes enable various analytical operations, including:
 - **Slicing:** Viewing a single dimension while keeping others constant.
 - **Dicing:** Focusing on a specific subset of data across multiple dimensions.
 - **Drilling Down:** Exploring more detailed data within a dimension.
 - **Rolling Up:** Summarizing data along a dimension.
- **Improves Performance:** By pre-aggregating data, data cubes enhance query performance, making it faster to retrieve and analyze large volumes of data.

In summary, a data cube is a powerful tool in data analysis that helps users visualize and understand complex datasets by organizing them in a structured, multi-dimensional format.

How can you apply data cubes in real-world analysis?

You can apply **data cubes in real-world analysis** in various industries and scenarios. Here are two relatable examples:

1. Retail Sales Analysis:

- **Scenario:** A retail company wants to analyze its sales data to understand customer purchasing behavior.
- **Application:**
 - **Dimensions:** The data cube might include dimensions such as **Product Category** (e.g., clothing, electronics), **Time** (e.g., year, quarter, month), and **Location** (e.g., store, region).
 - **Operations:** The company can slice the cube to view sales for a specific product category in a particular month, dice it to focus on sales of specific products across different regions, or roll up to see total sales by quarter.
 - **Insights:** This analysis helps the company identify trends, such as which products are popular during certain seasons or in specific locations, enabling better inventory management and targeted marketing strategies.

2. Healthcare Data Analysis:

- **Scenario:** A healthcare organization wants to analyze patient data to improve service delivery.
- **Application:**
 - **Dimensions:** The data cube might include dimensions like **Patient Demographics** (e.g., age, gender), **Treatment Type** (e.g., surgery, medication), and **Time** (e.g., year, month).
 - **Operations:** The organization can drill down to analyze treatment outcomes for specific age groups, slice the data to view trends in treatment types over time, or roll up to see overall patient satisfaction scores by year.

- **Insights:** This analysis helps the organization identify patterns in patient care, assess the effectiveness of treatments, and allocate resources more efficiently.

These examples illustrate how data cubes can be used to derive actionable insights from complex datasets in real-world scenarios.

What would happen if you didn't use materialized views?

If you didn't use **materialized views**, several challenges could arise in data management and analysis:

1. Increased Query Time:

- Without materialized views, every query would need to access the underlying tables directly, which can be time-consuming, especially for complex queries involving large datasets. This could lead to slower performance and longer wait times for users.

2. Higher Resource Consumption:

- Running complex queries repeatedly on large tables can consume significant system resources (CPU, memory, and I/O). This can affect the overall performance of the database and impact other operations.

3. Lack of Pre-Aggregated Data:

- Materialized views store pre-aggregated data, which simplifies and speeds up reporting and analysis. Without them, users would need to perform aggregations on-the-fly, making it harder to generate timely insights.

4. Increased Complexity in Query Writing:

- Users would need to write more complex SQL queries to retrieve the same information that could be easily accessed through a materialized view. This can lead to errors and make it harder for less experienced users to work with the data.

5. Potential for Outdated Data:

- If materialized views are not used, there may be a risk of working with outdated data, especially if the underlying tables are frequently updated. This can lead to inaccurate analysis and decision-making.

6. Difficulty in Data Replication:

- Materialized views can be used to replicate data for reporting purposes. Without them, maintaining consistent and up-to-date copies of data for analysis can become more challenging.

In summary, not using materialized views can lead to slower performance, increased resource usage, and more complex queries, ultimately hindering effective data analysis and decision-making.